

SHARIF
UNIVERSITY OF TECHNOLOGY



Computer Structure

Pre-Lab Report 2

Aria Fateh Kerdari
403110382

May 24, 2026

1 Introduction

This report details the design and implementation of three digital logic circuits: a 4-bit multiplier using the shift-and-add method, a sequence-controlled register operation circuit, and a generalized databus architecture for register transfers. The designs demonstrate fundamental computer architecture concepts, including datapath routing, control unit sequence generation, and shared bus architectures.

2 Combinational Multiplier

2.1 Objective

To design a 4-bit multiplier from scratch using Adders, Multiplexers (MUXes), and Barrel Shifters, implementing the hardware shift-and-add algorithm.

2.2 Inner Workings and Implementation

The multiplication of two 4-bit numbers (A and B) requires handling a maximum possible product of 8 bits. Therefore, all internal datapaths must be 8 bits wide to prevent data loss.

1. **Input:** Input A is zero-extended by padding it with four leading constant 0s, transforming it into an 8-bit operand. Input B is split into its individual bits (b_0, b_1, b_2, b_3) to act as control signals.
2. **Partial Products (MUXes):** Four 8-bit 2-to-1 MUXes generate the partial products. The select pin of each MUX is tied to a specific bit of B . The 0 input is an 8-bit constant zero, and the 1 input is the zero-extended A . If $b_i = 1$, A is passed; if $b_i = 0$, zero is passed.
3. **Shifting (Barrel Shifters):** The output of the i -th MUX is shifted left by i bits using Logical Shift Left Barrel Shifters.
4. **Adders:** Three 8-bit adders form an adder tree. Adder 1 sums the outputs corresponding to b_0 and b_1 . Adder 2 sums the outputs for b_2 and b_3 . Adder 3 sums the results of Adder 1 and Adder 2 to produce the final 8-bit product, R .

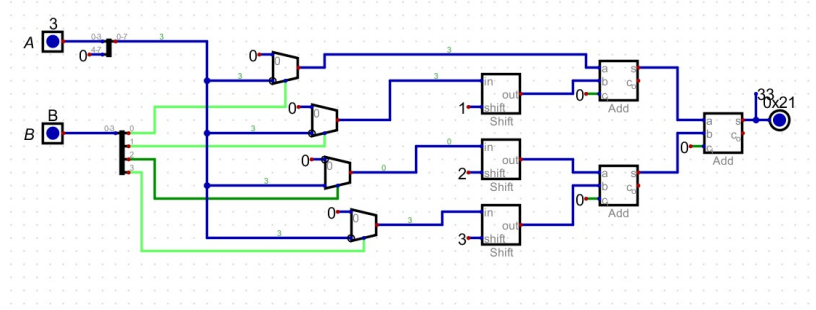


Figure 1: Schematic of the 8-bit extended shift-and-add multiplier.

3 Sequence-Controlled Register Operations

3.1 Objective

To construct a closed-system circuit with 8 registers capable of executing specific sequences (Load $R0$, Copy $R0 \rightarrow R1$, Copy $R3 \rightarrow R2$, and arithmetic operations between $R1$ and $R2$) using a Counter and an EEPROM as a control unit.

3.2 Inner Workings and Implementation

This circuit is divided into a Datapath and a Control Unit.

- **Datapath:** The system utilizes 8-bit registers ($R0$ to $R7$) and an 8-bit Adder/Subtractor. External inputs are routed to $R0$. The output of $R0$ is hardwired to the input of $R1$. The outputs of $R1$ and $R2$ feed into the A and B inputs of the Adder/Subtractor. The arithmetic result is stored in $R3$, whose output is hardwired back to the input of $R2$.
- **Control Unit:** An EEPROM paired with a counter generates microinstructions. We defined a 5-bit control word format:

$$[en - R0, en - R1, en - R2, en - R3, add - sub]$$

By pulsing the counter, the EEPROM cycles through addresses, outputting precise control signals. For example, to add $R1$ and $R2$ and store

the result in $R3$, the EEPROM outputs the control word 00010₂. This sets the `add-sub` to ADD (0) and asserts the Write Enable of $R3$, capturing the sum on the next clock edge.

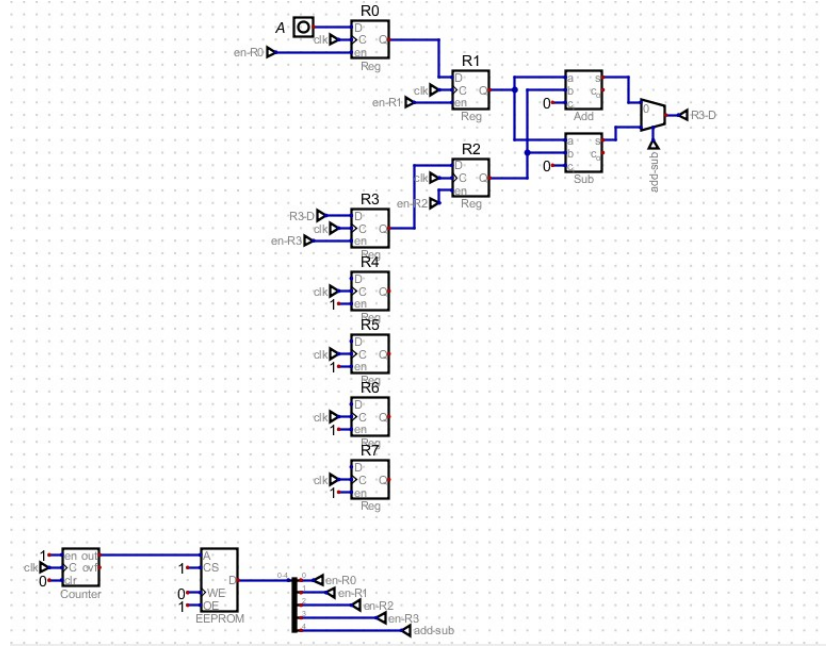


Figure 2: Register datapath controlled by EEPROM sequence.

4 General Register Transfer Architecture

4.1 Objective

To upgrade the register circuit to support generalized data transfers, allowing the contents of any source register (R_i) to be copied to any destination register (R_j) within a single clock cycle.

4.2 Inner Workings and Implementation

This requirement was fulfilled by implementing a shared `Databus` architecture, significantly reducing wiring complexity and increasing modularity.

1. **Source Selection (MUX):** An 8-bit 8-to-1 Multiplexer acts as the bus arbiter. The outputs (Q) of all registers ($R0$ to $R7$) are connected to the 8 inputs of this MUX. A 3-bit control signal, `Src_Sel`, chooses exactly one register's output to broadcast onto the shared 8-bit Databus.
2. **Destination Selection (Decoder):** The Databus routes to the Data Inputs (D) of all 8 registers simultaneously. To ensure only the correct register captures this data, a 3-to-8 Decoder is used. A 3-bit control signal, `Dst_Sel`, is decoded to assert exactly one output line. These lines are connected to the Write Enable pins of the registers.

4.3 Operation Walkthrough

To copy R_i to R_j , the control unit outputs i to `Src_Sel` and j to `Dst_Sel`. Before the clock edge, the MUX routes R_i 's data to the bus, and the Decoder holds R_j 's enable pin high. On the rising clock edge, only R_j updates its internal state with the value on the bus.

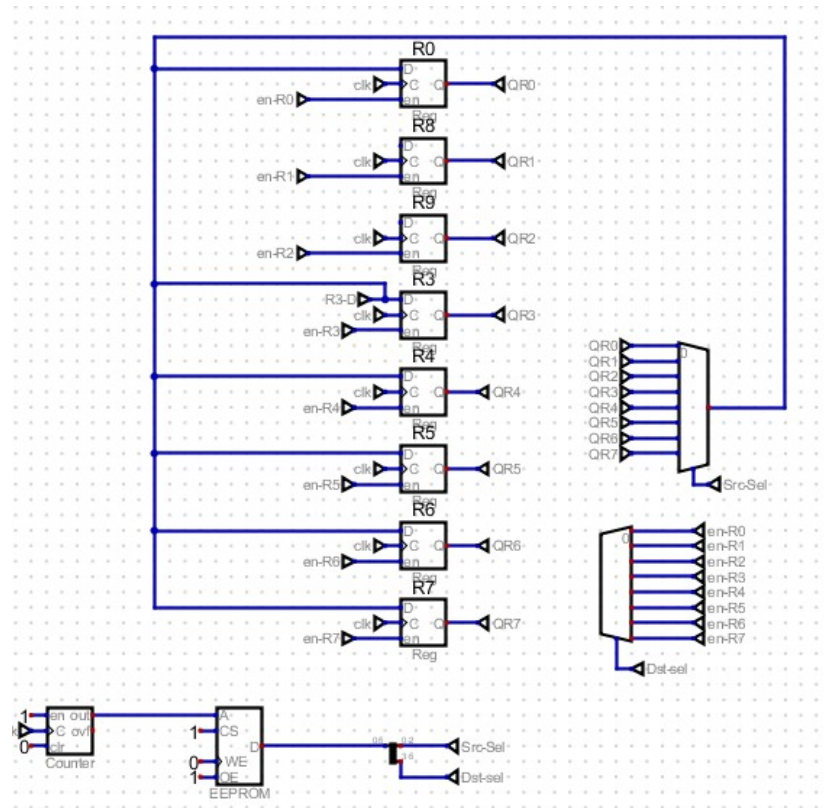


Figure 3: General Register Transfer using a Databus, MUX, and Decoder.